

JSON Web Signatures (JWS) in Depth



Scott Brady

Identity Consultant & Engineering Manager

@scottbrady91

www.scottbrady91.com



Overview



How do digital signatures work?

What are JSON Web Algorithms (JWA)?

What signing algorithm should I use?

- RSA
- ECDSA
- EdDSA
- HMAC



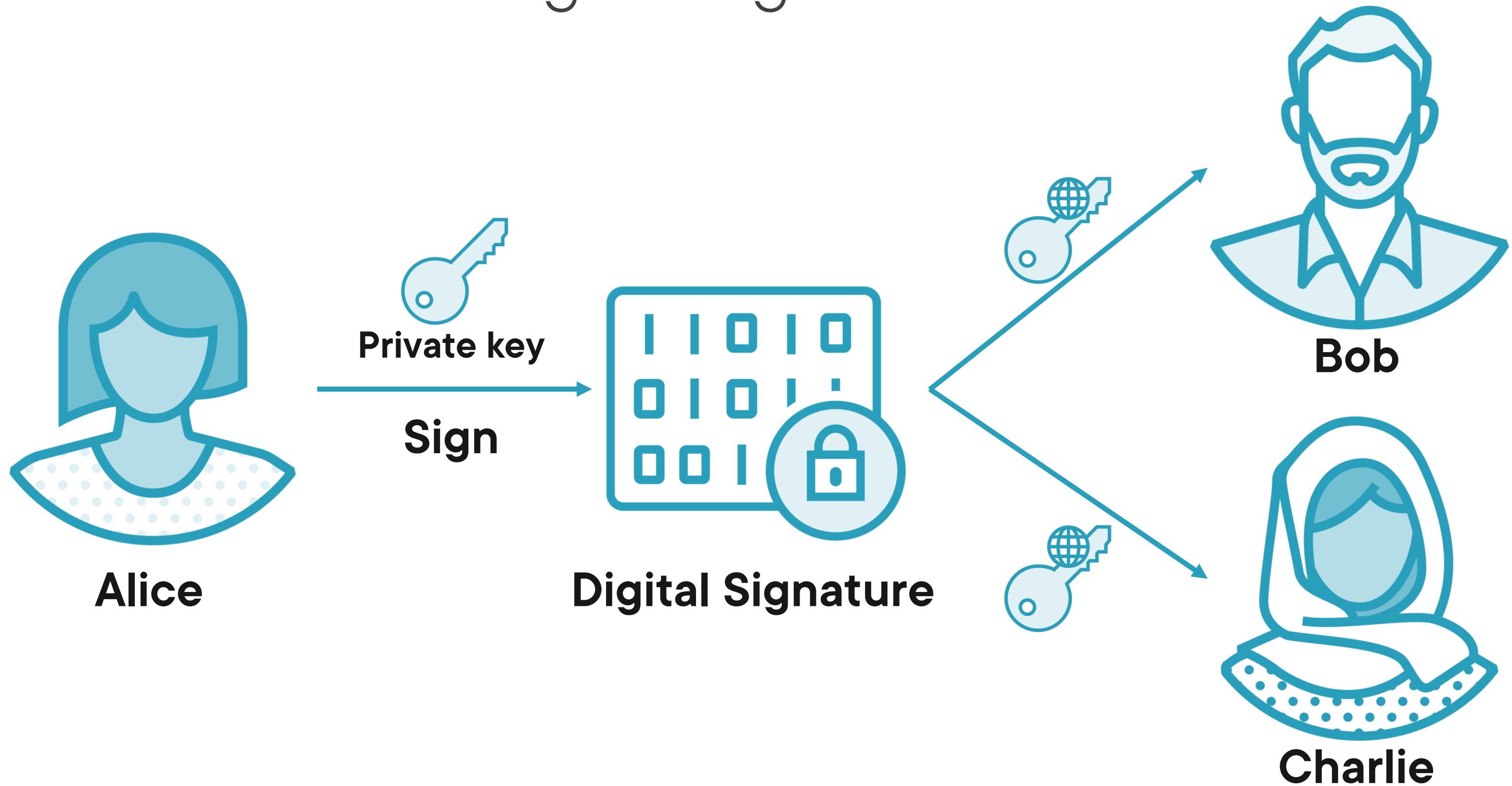
Understanding Digital Signatures and Signing Keys



Digital Signatures



Digital Signatures



Digital Signatures



Integrity



Authentication



Non-repudiation

JWT Signing

`ascii(encodedHeader + ' .' + encodedPayload) -> sign`

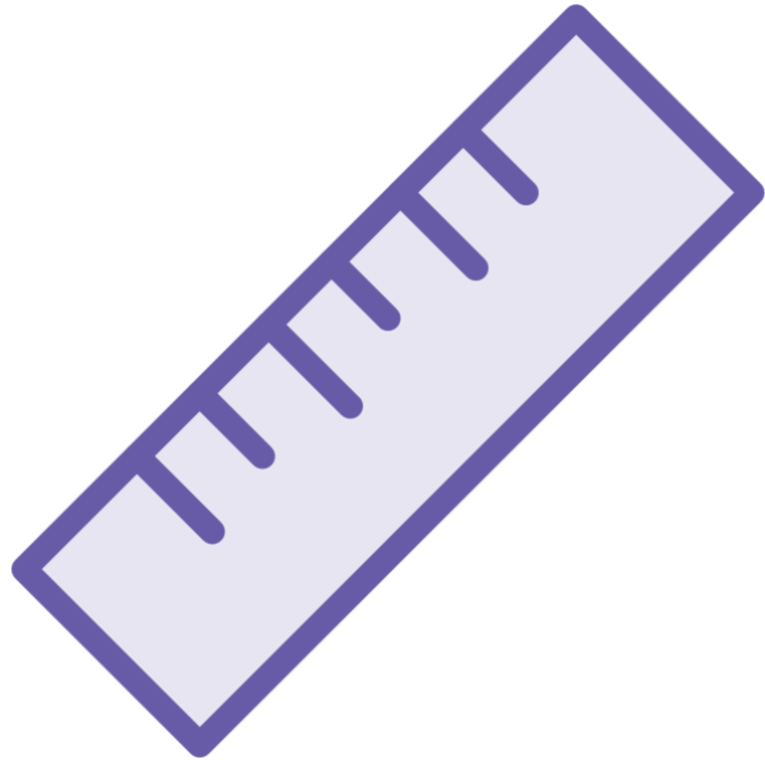


JWT Signing

`hash(ascii(encodedHeader + '.' + encodedPayload)) -> sign`



Hash and Then Sign



**Signing algorithm not
affected by JWT size**



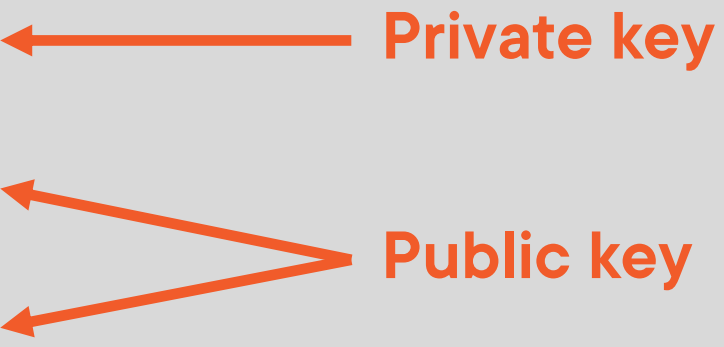
**Allows the use of
otherwise unavailable
signing algorithms**



Security benefits



```
{  
  "kty": "EC",  
  "alg": "ES256",  
  "crv": "P-256",  
  "use": "sig"  
  "d": "XY50IcfB08E-2lgm-vhJ4zcrBboko_0RXfUxcjdVaKc",  
  "x": "EVj0Bs7jPUeyfF2KdgaEP0i7yFbHWHbXU-t4jx750Fw",  
  "y": "dRfxMu14g9UJn5isKGhzvc3F3HS10n7MaFVdUAqyXTY",  
}
```



Private key

Public key

Signing Keys

Plain key bytes (base64url encoded).

X.509 Certificate

Version: 3
Serial number: 196501383010733369650670300464418191912298963055
Algorithm ID: SHA256withECDSA
Validity
Not Before: 18/09/2020 14:42:39 (dd-mm-yyyy hh:mm:ss) (200918144239Z)
Not After: 13/09/2021 14:42:39 (dd-mm-yyyy hh:mm:ss) (210913144239Z) ← Expiry
Issuer
C = AU, ST = Some-State, O = Internet Widgits Pty Ltd ← Issuer
Subject
C = AU, ST = Some-State, O = Internet Widgits Pty Ltd
Public Key ← Public key
Algorithm: EC
Curve Name: secp256r1
Length: 256 bits
pub: 04:70:7f:11:1b:e9:15:fe:7f:08:b3:ef:4b:29:11:d2:
d3:7c:af:e2:b5:ee:59:19:fc:ac:6d:a9:4c:fd:99:67:
52:35:99:f8:2c:24:f9:a1:1c:2a:41:11:d8:65:a8:67:
a1:62:c3:73:dc:3a:54:33:63:c7:46:1c:c7:64:90:78:
46
Certificate Signature ← Certificate signature
Algorithm: SHA256withECDSA
r: 00:c5:7e:cd:e9:ef:b6:35:fb:b0:47:bd:2e:dc:05:f4:
9b:1e:59:3f:db:90:85:da:ef:3a:2f:2a:0c:11:d2:30:
ca
s: 85:da:ef:3a:2f:2a:0c:11:d2:30:ca:02:20:44:1e:9d:
43:2d:4d:f6:1f:39:4f:05:c1:3d:d6:64:ee:12:e4:90:
79:8e:f3:e0:f6:10:a7:95:73:e3:37:06:82



JSON Web Algorithms (JWA) and JWS



JSON Web Algorithms (JWA)

(RSASSA-PKCS1-v1_5)

RS

signature algorithm

(SHA-256)

256

hashing algorithm



Hashing Algorithms

RS256

RS384

RS512

RS1



Which Hashing Algorithm Should You Use?



Use a hashing algorithm from the SHA-2 family



“256” refers to the length of the generated hash



Use a minimum of 128-bit security



Never use SHA-1!



Always validate the key.



```
{  
  "typ": "JWT",  
  "kid": "f7811b04be0844c0b8c77250504623d9", // is this key...  
  "alg": "RS256" // ...allowed to be used for this algorithm?  
}
```

Key and Algorithm Validation

Store keys & algorithm pairs

Do not accept signatures using a key for an unexpected algorithm

Do not accept unknown algorithms or keys

```
{  
  "typ": "JWT",  
  "kid": "f7811b04be0844c0b8c77250504623d9",  
  "alg": "RS256",  
  "x5t#S256": "6979da8247c3080de96e861e9f000a22d6120170a3982bea4e9f054598f6453f"  
}
```

Key and Algorithm Validation

“x5t” or “x5t#S256” headers for certificate thumbprints.

Beware "alg": "none"



Signing Algorithm:
RSASSA-PKCS1-v1_5 (e.g. RS256)



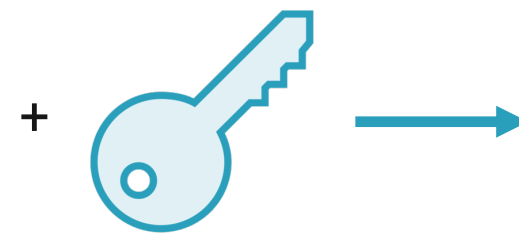
RS256

RSASSA-PKCS1-v1_5 using SHA-256



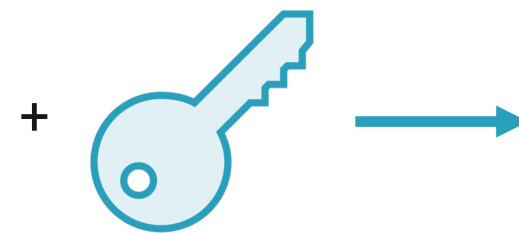
Deterministic Signatures

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsI
mtpZCI6IjBkN2ZiY2MwMmZiYmQxOGM3YzY2Nj
VhMGJiYjYjYmI4In0.eyJzdWIiOiIxMjM0NTY
3ODkwIiwibmFtZSI6Imlnbn3R0IiwiaWF0Ijox
NjU4MzIwMzIzZfQ



PY0h0yAfLVqUqttJtuxV0WhIUMfLw47NHrsL-Go3aJSn4pMILStxKvjU40
DBvEe17bbE81KRhFzrIb6EtBL-EKBBE4xVrP1GfJCQWa2uhTRg6u51b1h0
ZFahWG8cmpgdZ1Qubcoasyr1aK9YaZdA0sgRmZQsr8vhGm6voyNCsx4YdM
BquRzM9CnI3Hsi50mkj-ctl_nA-vBzqdA0HTuiXR_BwXNxjHt5qk5HpYGU
xcfzceahtXJhaSSotS8x5Bzh5rJpK2pe8_tuC6QE_mnQz3MdzfcSa0orsz
IZFLuEICp1yXVFN2x4zVdjd1vEtYsNmiXJBE0G5oMmgV2n7n03Vw

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsI
mtpZCI6IjBkN2ZiY2MwMmZiYmQxOGM3YzY2Nj
VhMGJiYjYjYmI4In0.eyJzdWIiOiIxMjM0NTY
3ODkwIiwibmFtZSI6Imlnbn3R0IiwiaWF0Ijox
NjU4MzIwMzIzZfQ



PY0h0yAfLVqUqttJtuxV0WhIUMfLw47NHrsL-Go3aJSn4pMILStxKvjU40
DBvEe17bbE81KRhFzrIb6EtBL-EKBBE4xVrP1GfJCQWa2uhTRg6u51b1h0
ZFahWG8cmpgdZ1Qubcoasyr1aK9YaZdA0sgRmZQsr8vhGm6voyNCsx4YdM
BquRzM9CnI3Hsi50mkj-ctl_nA-vBzqdA0HTuiXR_BwXNxjHt5qk5HpYGU
xcfzceahtXJhaSSotS8x5Bzh5rJpK2pe8_tuC6QE_mnQz3MdzfcSa0orsz
IZFLuEICp1yXVFN2x4zVdjd1vEtYsNmiXJBE0G5oMmgV2n7n03Vw

RSA Signatures



Signature length == key length



Use a minimum key length of 3072 (2048 for legacy systems)



Wide-spread use of RS256



“Unlike the first attack that broke the encryption algorithm completely, the second attack is an implementation attack [against signature validation]. This means that if the signature scheme is implemented correctly (according to the specification), the attack does not work.

Yet, it was shown in 2019 that many open source implementations of RSA PKCS#1 v1.5 for signatures actually fell for that trap and mis-implemented the standard, which enabled different variants of Bleichenbacher’s forgery attack to work!”

David Wong, *Real-World Cryptography*



“Although no attacks are known against RSASSA-PKCS1-v1_5, in the interest of increased robustness, RSASSA-PSS is **REQUIRED** in new applications.”

RFC 8017 - PKCS #1: RSA Cryptography Specifications Version 2.2



Signing Algorithm: RSASSA-PSS (e.g. PS256)



PS256

RSASSA-PSS using SHA-256 with MGF1 with SHA-256



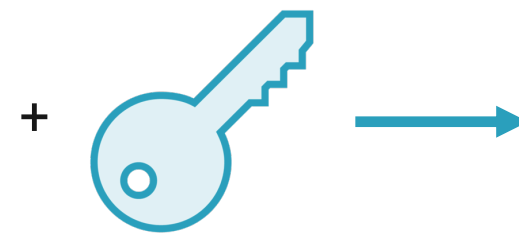
RSA-PSS

Algorithm	Description
PS256	RSASSA-PSS using SHA-256 and MGF1 with SHA-256
PS384	RSASSA-PSS using SHA-384 and MGF1 with SHA-384
PS512	RSASSA-PSS using SHA-512 and MGF1 with SHA-512



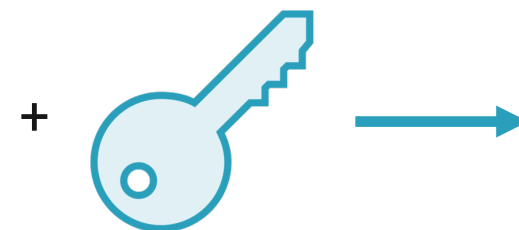
Probabilistic Signatures

eyJ0eXAiOiJKV1QiLCJhbGciOiJQUzI1NiIsI
mtpZCI6ImY2M2NhOGNlZjUxZGIwOTdlNjkwNT
Y4ZTEuYjY4NjZjIn0.eyJzdWIiOiIxMjM0NTY
3ODkwIiwibmFtZSI6Imlnbn3R0IiwiaWF0Ijox
NjU4MzIwMzIzfQ



TQGnYnr_MVwP--4dnjm15MJrLe1wnB-H-5Qoem03C2z8J_qnt8ff7hJF2R
e2Gz8K1UE1Tu2gS9gvFcBeXpkzqQw2UnqqTM-qaxkxMgIJPDDFDT_5adVN
15mppZPUNxEQ3IDVSmbHwi0qp0CY96fdym_Ysq3mCyj14BNEzOwKktSSuX
ERgPpWkaNPuI6jwhVL1vSqng97W0uPAuKlgXdtCub9ef40yJ9ET8SB-0kb
zqHbsnJ3_6KeQQL6CvB8moBcx0WdIz0yv0dhxih3aW3hngrSRAoydK_d4_
eYfCrb1QFJF44wJ3C9JNRV6V_CnEG1JqGRA4JzoumVCz9LSHVRHA

eyJ0eXAiOiJKV1QiLCJhbGciOiJQUzI1NiIsI
mtpZCI6ImY2M2NhOGNlZjUxZGIwOTdlNjkwNT
Y4ZTEuYjY4NjZjIn0.eyJzdWIiOiIxMjM0NTY
3ODkwIiwibmFtZSI6Imlnbn3R0IiwiaWF0Ijox
NjU4MzIwMzIzfQ



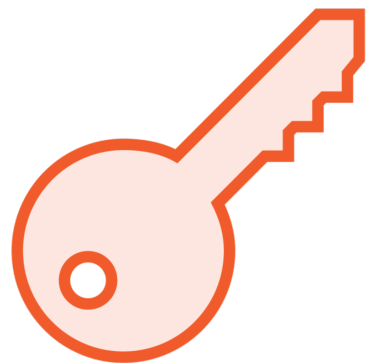
ehDoeHKtaElFm0d9Kxr_swzyvXMlp8ywydGNTS3t3uCSJXrjFuVLh_Dzdu
Xg_PEdW0sdVme-uzH7gBt0Fn2l2Lb10V00e50ZlCtaG4XKGzmGpDB0u07Z
txKqKEDGjX_Lx0KqvaF8jReVE0czG4DA73Ra4YYs1FuBFX36NpnYnXSTOo
i3G0xy1id1fXwSkwGhvxC9xq1WwxcabcIWUo4lRi1thwYh2NeqDz7qLld0
5U45J1NhBkwAUza1I-vHvj66oF5kM6uGu0b7_bbd2TYv4nYifCvW-gj0kF
dVbbR9KCpvaC8yMC_crp_QGAmo3Gg07kyCEyUk_oREc6-SPmgRCg



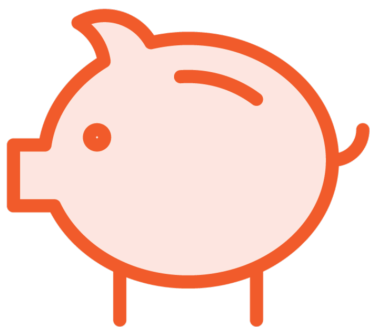
RSA-PSS Signatures



Probabilistic in a good way



Uses the same keys as RSASSA-PKCS1-v1_5



Required by Open Banking & Financial-grade API



Signing Algorithm: ECDSA (e.g. ES256)



ES256

ECDSA using P-256 and SHA-256

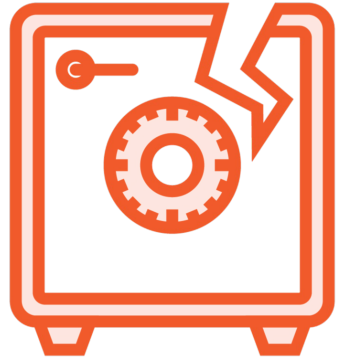


ECDSA

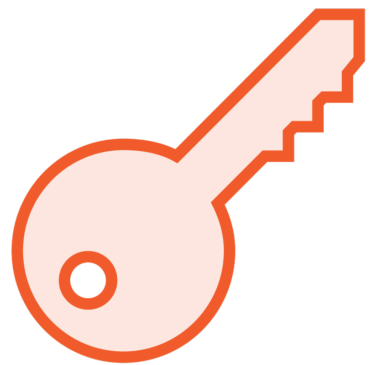
Algorithm	Hashing algorithm	Curve
ES256	SHA-256	P-256 (secp256r1, aka prime256v1)
ES384	SHA-384	P-384 (secp384r1)
ES512	SHA-512	P-521 (secp521r1)



ECDSA



Harder to break than RSA



Shorter keys & shorter signatures



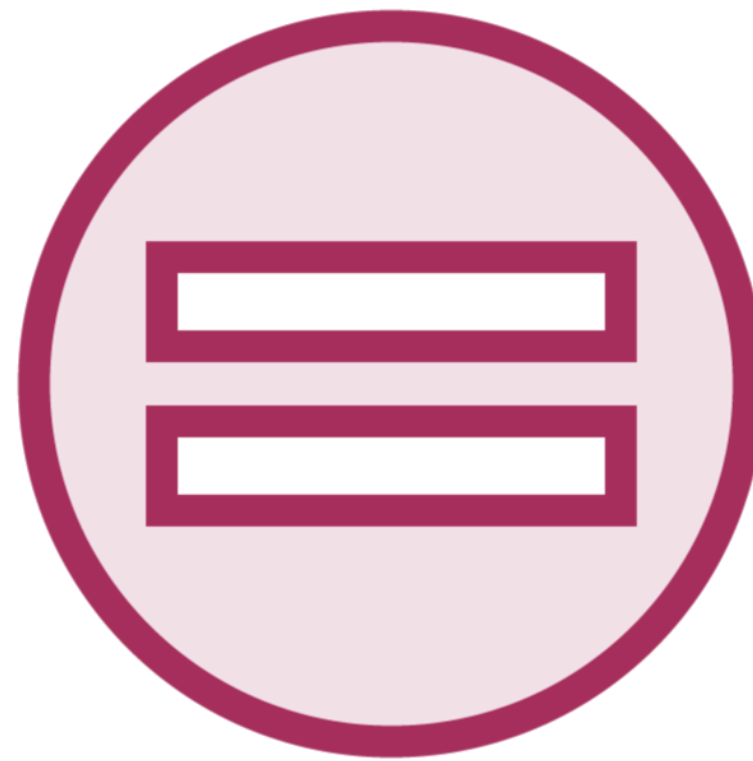
Probabilistic in a bad way



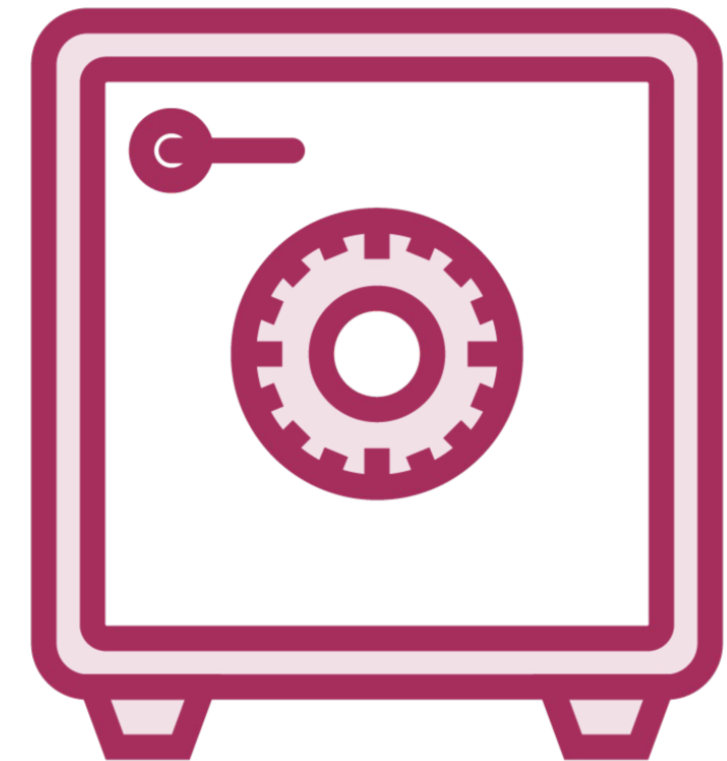
ECDSA Attacks



**Signature generation:
prevent nonce re-use**



**Use deterministic
ECDSA
(RFC 6979)**

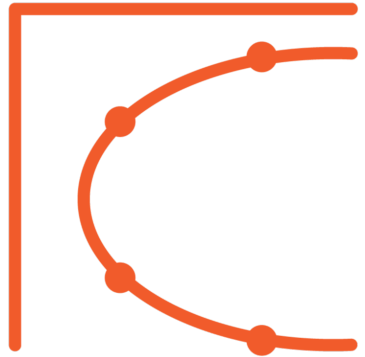


**Signature validation:
use an up-to-date
implementation**

ECDSA is difficult to implement.
Consider using EdDSA.



ES256K



Use of non-NIST curves (secp256k1)



A few bits weaker than ES256



Used in Bitcoin, Ethereum, and FIDO2



Signing Algorithm: EdDSA



EdDSA

“An EdDSA signature algorithm was used”



```
{  
  "kty": "OKP",  
  "alg": "EdDSA",  
  "crv": "Ed25519",  
  "x": "60mR98SQ1HUSELeIu7TeJBTLRG10q1cDLU4AJjQdqMQ"  
}
```

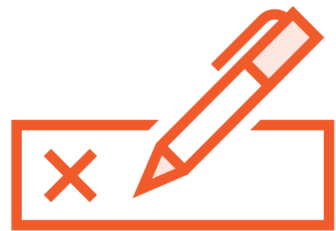
Pre-agreed Keys Only

Curve is known to the verifier ahead of time.

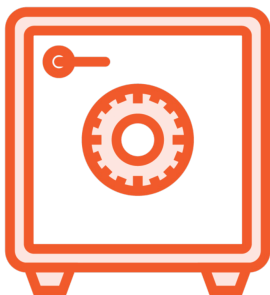
EdDSA – Twisted Edwards Curves!



Fast for both signature creation and validation



Short signatures



Modern cryptography that side-steps old attacks



Deterministic in a good way



Supported EdDSA Variants

Curve	Hashing algorithm	Key Length	Security
Ed25519	SHA-256	32-bytes	128-bit
Ed448	SHAKE256	57-bytes	224-bit



Digital Signatures vs. Message Authentication Codes (e.g. HS256)



HS256

HMAC using SHA-256



Hash-Based Message Authentication Code (HMAC)



Hash-Based Message Authentication Code (HMAC)



HMAC Security



Integrity



Authentication



Non-repudiation

Consider your use case.



Summary



How do digital signatures work?

What are JSON Web Algorithms (JWA)?

What signing algorithm should I use?

- EdDSA > ES256 or PS256 > RS256
- HMAC probably isn't right for you



IANA Registry

Algorithm Name 	Algorithm Description 	Algorithm Usage Location(s) 	JOSE Implementation Requirements 	Change Controller 	Reference 
HS256	HMAC using SHA-256	alg	Required	[IESG]	[RFC7518, Section 3.2]
HS384	HMAC using SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.2]
HS512	HMAC using SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.2]
RS256	RSASSA-PKCS1-v1_5 using SHA-256	alg	Recommended	[IESG]	[RFC7518, Section 3.3]
RS384	RSASSA-PKCS1-v1_5 using SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.3]
RS512	RSASSA-PKCS1-v1_5 using SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.3]
ES256	ECDSA using P-256 and SHA-256	alg	Recommended+	[IESG]	[RFC7518, Section 3.4]
ES384	ECDSA using P-384 and SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.4]
ES512	ECDSA using P-521 and SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.4]
PS256	RSASSA-PSS using SHA-256 and MGF1 with SHA-256	alg	Optional	[IESG]	[RFC7518, Section 3.5]
PS384	RSASSA-PSS using SHA-384 and MGF1 with SHA-384	alg	Optional	[IESG]	[RFC7518, Section 3.5]
PS512	RSASSA-PSS using SHA-512 and MGF1 with SHA-512	alg	Optional	[IESG]	[RFC7518, Section 3.5]
none	No digital signature or MAC performed	alg	Optional	[IESG]	[RFC7518, Section 3.6]

<https://www.iana.org/assignments/jose/jose.xhtml#web-signature-encryption-algorithms>

